

1. Prove $\left(\frac{(n(n+1))^2}{4} - \frac{n^2(n^2 + 1)}{4} + 78\right)$ belongs to $O(n^3)$.

$$\left(\frac{(n(n+1))^2}{4}\right)^2 = \frac{(n^2 + n)^2}{4} = \frac{n^4 + 2n^3 + n^2}{4}$$

$$\frac{n^2(n^2 + 1)}{4} = \frac{n^4 + n^2}{4}$$

$$\frac{n^4 + 2n^3 + n^2}{4} - \frac{(n^4 + n^2)}{4} + 78$$

$$\frac{n^4}{4} - \frac{n^4}{4} + \frac{2n^3}{4} + \frac{n^2}{4} - \frac{n^2}{4} + 78$$

$$0 + \frac{n^3}{2} + 0 + 78$$

$$\frac{n^3}{2} + 78$$

Since the lead term is n^3 and coefficients like 78 do not affect time, the expression clearly belongs to $O(n^3)$. There is no need for any constants (though in this case it is 1) for cn^3 , it is obviously indicated that the equation does indeed belong to $O(n^3)$.

2. Prove or disprove $2^{(2^n)}$ belongs to $O(2^{2n})$.

PROOF BY CONTRADICTION!

Assume that $2^{(2^n)}$ belongs to $O(2^{2n})$, which implies that $2^{(2^n)} \leq c2^{2n}$

Then, $2^{(2^n)}/2^{2n}$ in order to isolate c .

$$2^{(2^n)}/2^{2n} = 2^{(2^n - 2n)}$$

$$2^{(2^n - 2n)} \leq c \text{ CONTRADICTION!}$$

Left side grows faster than right because right is a constant and for some arbitrary n , it will grow larger than c . Therefore, it is a contradiction and expression does not belong to $O(2^{2n})$.

3. Prove that any function that is in $O(\log_2 n)$ is also in $O(\log_3 n)$.

Using change of base log theorem: $\log_2(n) = \log_3(n)/\log_3(2)$.

$$- f(n) = \log_2(n)$$

$$\text{So, } 0 \leq f(n) \leq c(\log_3(n)/\log_3(2))$$

Since $\log_3(2)$ is a constant, $c' = c/\log_3(2)$

Therefore, $f(n)$ or $\log_2(n) \leq c'(\log_3(n))$ and shows that for all arbitrary n , $\log_2(n)$ belongs to $O(\log_3(n))$ and that the statement is proven true.

4. Prove that if $f_1(n)$ belongs to $O(g_1(n))$ and $f_2(n)$ belongs to $O(g_2(n))$, then $f_1(n) + f_2(n)$ belongs to $O(g_1(n) + g_2(n))$.

$f_1(n)$ belongs to $O(g_1(n))$

Therefore, $f_1(n) \leq c(O(g_1(n)))$

$f_2(n)$ belongs to $O(g_2(n))$

Therefore, $f_2(n) \leq c(O(g_2(n)))$

$f_1(n) + f_2(n) \leq C(c(g_1(n)) + c(g_2(n)))$

$(f_1(n) + f_2(n)) / (c(g_1(n)) + c(g_2(n))) \leq C$

Denominator will grow at a faster rate than the numerator as all functions have n as runtime/growth rate and the constants already applied to the $g(n)$ functions make it larger than $f(n)$. Therefore for all $C \geq 1$, the statement is proven true and $f_1(n) + f_2(n)$ belongs to $O(g_1(n) + g_2(n))$.

```
for (i = 1; i <= n; i++)
{
    for (j = n; j >= 1; j--)
    {
        for (k = 1; k <= i + j; k++)
        {
            // some constant number of atomic operations
        } // end k
    } // end j
} // end i
```

5. Derive the runtime of the following loop structure as a function of n and determine its Big-O upper bound. You must show the derivation of the end result. Simply stating the final answer without any derivation steps will result in zero points. Assume atomic operations take unit time.

- 1 is the minimum value in this for loops in terms of this iteration.
- Outermost loop iterates from 1 to n , therefore it runs n times.
- Middlemost loop iterates from n to 1, therefore it runs n times.
- Innermost loop iterates from 1 to $i + j$, therefore it runs $i + j$ times.
 - Not compatible with n yet, so must be worked further.

Total # of operations = $(i = 1)(n) \sum (j = 1)(n) \sum (i + j)$

Which is also equivalent to: $(i = 1)(n) \sum (j = 1)(n) \sum i + (i = 1)(n) \sum (j = 1)(n) \sum j$

- J is constant for inner sum over i and i is constant for inner sum over j .

$$(i = 1)(n) \Sigma n * i + (j = 1)(n) \Sigma n * j$$

$$\bullet \sum_{i=1}^n i = \frac{n(n+1)}{2} \quad - \quad \text{By this theorem it can be further simplified.}$$

$$\begin{aligned} & n * (n(n+1))/2 + n * (n(n+1))/2 \\ &= n * (n+1) * n \\ &= n^3 + n^2 \end{aligned}$$

- Since Run time only focuses on highest power/exponent, n^2 can be discarded.

Therefore, the run time of the indicated for loop is $O(n^3)$ assuming that constant operations are atomic and not a factor.